Lecture 2: Sparse and Dense Indexing

Information Retrieval — Lecture Notes

Avishek Anand
Information Retrieval Course

February 13, 2026

Abstract

These lecture notes accompany Lecture 2 of the Information Retrieval course, covering the two foundational indexing paradigms: **sparse (inverted) indexing** and **dense (vector) indexing**. We present formal definitions, worked examples, algorithmic descriptions, and practical guidance for building production retrieval systems. Topics include text preprocessing, index compression, approximate nearest neighbor search, and design decisions for choosing between indexing strategies.

Contents

1 The Modern IR Landscape **3**
1.1 From 10 Blue Links to RAG . . . 3
1.2 The Interface Changed, the Engine Did Not . . . 4
1.3 Why Indexing Matters . . . 4
1.4 Course Context . . . 4

2 Sparse Indexing: The Classical Approach **5**
2.1 The Inverted Index . . . 5
2.1.1 Worked Example: Building an Inverted Index . . . 5
2.2 The Inverted Index as a Sparse Matrix . . . 6
2.3 Complexity Analysis . . . 6
2.4 Posting List Formats . . . 6
2.5 Score Aggregation and the Vector Space Model . . . 7
2.6 Text Preprocessing Pipeline . . . 7
2.6.1 Tokenization . . . 8
2.6.2 Stemming vs. Lemmatization . . . 8
2.6.3 Stop Words . . . 8
2.6.4 Preprocessing Depends on Representation . . . 8
2.7 Index Compression . . . 8
2.7.1 Gap Encoding . . . 9
2.7.2 Variable Byte (VB) Encoding . . . 9
2.7.3 Compression in Practice . . . 9
2.8 PyTerrier: Sparse Indexing in Practice . . . 9

3 Dense Vector Indexing for Neural Retrieval **10**
3.1 The Shift to Dense Vectors . . . 10
3.2 The ANN Search Problem . . . 10
3.3 IVF: Inverted File Index for Vectors . . . 11
3.3.1 IVF Construction . . . 11

3.3.2 IVF Search . . . 12
3.4 Product Quantization (PQ) . . . 12
3.4.1 PQ Query Processing in Detail . . . 12
3.4.2 Worked Example: PQ Query Processing . . . 14
3.5 IVF + PQ: The Production Standard . . . 16
3.6 FAISS: Dense Indexing in Practice . . . 16

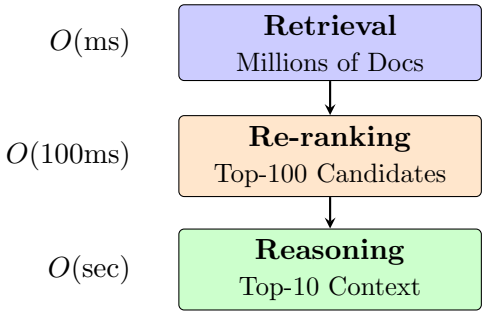
4 Design Decisions: When to Use What 17
4.1 Decision Matrix . . . 17
4.2 Case Study: Academic Paper Search (100M+) . . . 17
4.3 Common Pitfalls . . . 17
4.4 The Full Pipeline . . . 18
4.5 Production Checklist . . . 18

5 Summary and Next Steps 18
5.1 What We Covered . . . 18
5.2 Key Takeaways . . . 18
5.3 Connection to Upcoming Lectures . . . 19

1 The Modern IR Landscape

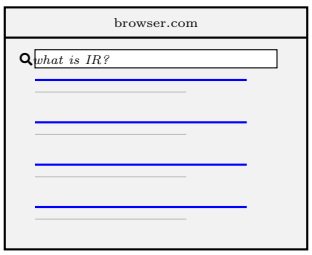
Modern information retrieval has undergone a fundamental transformation. While the user-facing interface has evolved from ranked lists of "blue links" to synthesized natural-language answers, the underlying retrieval engine remains remarkably consistent.

1.1 From 10 Blue Links to RAG

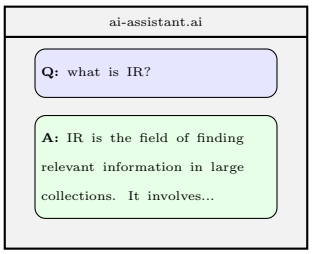


Definition 1.1 (Traditional Retrieval)**.** Traditional retrieval is a paradigm in which a user issues a keyword query and the system returns a ranked list of documents. The user is responsible for reading, evaluating, and synthesizing the returned results.

Traditional search engines (Google, Bing, Yahoo circa 2000s) exemplify this model. The user receives a ranked list of documents and must click through links and read to find the answer.



Definition 1.2 (Retrieval-Augmented Generation (RAG))**.** Retrieval-Augmented Generation (RAG) is a paradigm in which a retrieval system first fetches relevant passages from a document collection, augments a large language model (LLM) prompt with these passages, and the LLM generates a synthesized natural-language answer [4].

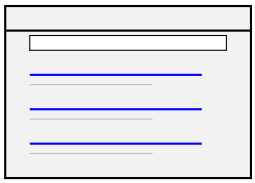


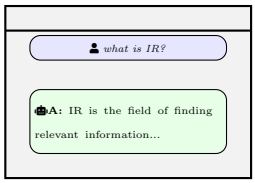
The RAG process consists of three stages:

1. **Retrieve** relevant passages from an index.
2. **Augment** the LLM prompt with retrieved content.
3. **Generate** a final answer using the LLM.

Modern systems such as ChatGPT (with browsing), Perplexity, and Bing Chat follow this pattern.

1.2 The Interface Changed, the Engine Did Not



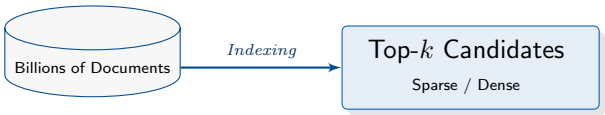


Key Takeaway

Retrieval now serves models instead of users. In RAG systems, the retrieval component acts as the LLM's "eyes" into the document collection. The underlying components—indexing, retrieval, and relevance scoring—remain the same whether we serve 10 blue links or a generated answer.

1.3 Why Indexing Matters

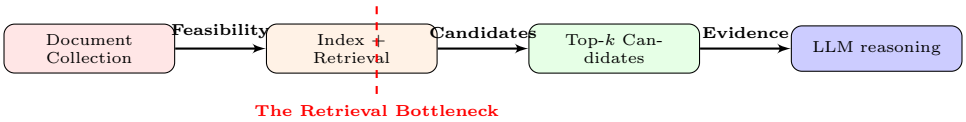
Indexing is the process of organizing a document collection into a data structure that enables efficient retrieval. Without indexing, every query would require scanning the entire collection.



Three reasons make indexing indispensable:

1. **Scale:** We cannot score every document for every query in real time.
2. **Latency:** Users and LLMs expect sub-second responses.
3. **Feasibility:** We must reduce millions of documents to hundreds of candidates.

The *recall ceiling* of an indexing system is the maximum recall achievable by any downstream component. If a relevant document is not returned by the candidate generation stage, no re-ranker or LLM can recover it [9].



Key Takeaway

Indexing is not just an optimization—it is the bridge to feasibility. If retrieval misses a document, the LLM cannot reason about it.

1.4 Course Context

This lecture sits in the following sequence: (1) Introduction to IR: VSM & Boolean Retrieval; (2) **Modern Indexing for RAG** ← Today; (3) Ranking: BM25, Query Expansion, Language Models; (4) Embeddings & Transformer Re-rankers; (5) Dense Retrieval Deep Dive; (6) Learning to Rank.

2 Sparse Indexing: The Classical Approach

2.1 The Inverted Index

The inverted index is the foundational data structure of text retrieval, powering Google Search, Elasticsearch, OpenSearch, PostgreSQL full-text search, and the sparse component of virtually every RAG system in production.

Definition 2.1 (Inverted Index)**.** An *inverted index* is a data structure that maps each term t in the vocabulary V to a *posting list*—an ordered list of document identifiers (and optionally term frequencies and positions) in which term t appears:

$$\text{InvertedIndex} : V \to \{(d_1, \text{payload}_1), (d_2, \text{payload}_2), \ldots\}$$

where each d_i is a document identifier and payload_i may contain the term frequency $\text{tf}(t, d_i)$, positional information, or other metadata [6].

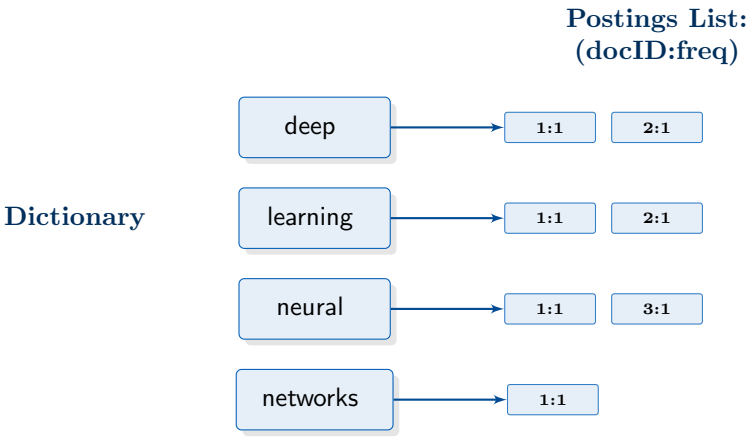
Why still relevant in the age of neural networks?

1. **Exact matching:** Finds "iPhone 15 Pro Max" precisely.
2. **Speed:** 5–10 ms queries over millions of documents.
3. **Explainability:** Can inspect which terms matched.
4. **Hybrid search:** Complements dense retrieval.

2.1.1 Worked Example: Building an Inverted Index

Example 2.1 (Constructing an Inverted Index)**.** Consider three documents:

- **Doc 1:** "neural networks for deep learning"
- **Doc 2:** "deep learning applications"
- **Doc 3:** "neural machine translation"



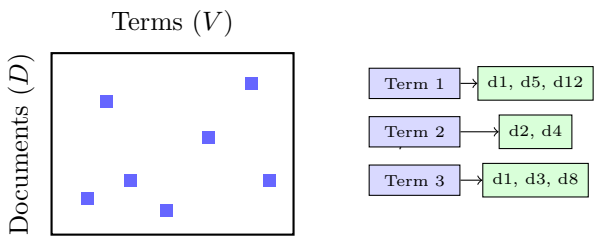
After tokenization and lowercasing, the inverted index is:

Term	Posting List
neural	$[1, 3]$
networks	$[1]$
for	$[1]$
deep	$[1, 2]$
learning	$[1, 2]$
applications	$[2]$
machine	$[3]$
translation	$[3]$

Query Processing for "neural learning":
1. Look up "neural" $\rightarrow [1, 3]$.
2. Look up "learning" $\rightarrow [1, 2]$.
3. Intersect: $[1, 3] \cap [1, 2] = [1]$.
4. Result: Document 1 contains both terms.

2.2 The Inverted Index as a Sparse Matrix

Definition 2.2 (Document-Term Matrix)**.** The *document-term matrix* $\mathbf{M} \in \mathbb{R}^{D \times |V|}$ has rows for documents, columns for vocabulary terms, and entries $\mathbf{M}_{i,j}$ containing the weight (TF-IDF, BM25) of term j in document i. This matrix is extremely sparse—typically >99.9% zeros.



Key Takeaway

The inverted index is a **Compressed Sparse Column (CSC)** storage format of the document-term matrix. We store only non-zero entries, avoiding the full $D \times |V|$ cost.

2.3 Complexity Analysis

Proposition 2.1 (Complexity of Inverted Index Retrieval)**.** *Given N documents with average length M, a query of k terms, and average posting list length p:*
- ***Naive scan:*** $O(N \cdot M)$.
- ***Inverted index:*** $O(k \cdot p)$, *where typically $k \ll N$ and $p \ll N$.*

Example 2.2 (Speed Comparison)**.** For 10M documents $\times$ 500 words: naive requires 5×10^9 comparisons. For query "neural learning" ($k = 2$, $p \approx 1000$): inverted index needs $\sim$2000 comparisons—$\sim$**1000**$\times$ **faster**.

2.4 Posting List Formats

Definition 2.3 (Posting List Levels)**.**

1. **Level 1 — DocID Only (Boolean Retrieval):** $t \rightarrow [d_1, d_2, \ldots]$. Supports AND/OR/NOT but no ranking.

2. **Level 2 — With Frequencies (Ranked Retrieval):** $t \rightarrow [(d_1, \mathrm{tf}_1), (d_2, \mathrm{tf}_2), \ldots]$. Supports TF-IDF, BM25. **Standard in practice.**

3. **Level 3 — With Positions (Phrase Queries):** $t \rightarrow [(d_1, \mathrm{tf}_1, [\mathrm{pos}_{1,1}, \ldots]), \ldots]$. Supports exact phrases and proximity scoring. 2–3$\times$ larger.

Example 2.3 (Posting List Formats)**.** For the term "neural":
- **Level 1:** `neural: [1, 3, 5, 12, 27, 89]`
- **Level 2:** `neural: [(1,tf=2), (3,tf=1), (5,tf=3), ...]`
- **Level 3:** `neural: [(1,tf=2,pos=[5,42]), (3,tf=1,pos=[8]), ...]`

2.5 Score Aggregation and the Vector Space Model

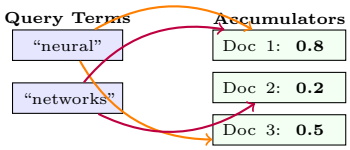
Definition 2.4 (Vector Space Model Scoring)**.** In the Vector Space Model [10], relevance is:

$$\text{score}(q, d) = \sum_{t \in q \cap d} w(t, q) \times w(t, d) \tag{1}$$

where $w(t, q)$ and $w(t, d)$ are term weights in query and document (e.g., TF-IDF).

The engine uses *accumulators*:

1. Initialize $\text{Acc}[d] = 0$ for all d.
2. For each query term t, traverse its posting list; for each (d, tf): $\text{Acc}[d] \mathrel{+}= w(t, q) \times w(t, d)$.
3. Return documents sorted by $\text{Acc}[d]$.

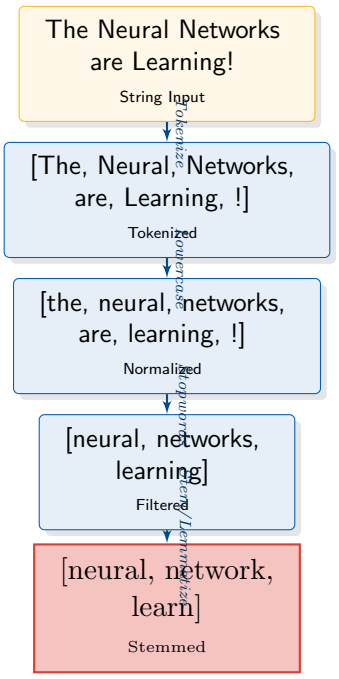


Key Takeaway

Sparse indexing computes scores **only for documents containing at least one query term**. All others are never touched.

2.6 Text Preprocessing Pipeline

The same preprocessing must be applied to both documents and queries.



Example 2.4 (Preprocessing Pipeline)**. Raw:** "The Neural Networks are Learning!"

1. **Tokenize:** ["The", "Neural", "Networks", "are", "Learning", "!"]
2. **Lowercase:** ["the", "neural", "networks", "are", "learning", "!"]
3. **Remove stop words:** ["neural", "networks", "learning"]
4. **Stem (Porter):** ["neural", "network", "learn"]

2.6.1 Tokenization

Definition 2.5 (Tokenization)**.** *Tokenization* segments text into discrete tokens (words or subwords), defining the index vocabulary.

Language-specific challenges: English contractions ("don't"), German compounds ("Informationssystem"), Chinese/Japanese (no whitespace).

2.6.2 Stemming vs. Lemmatization

Definition 2.6 (Stemming)**.** *Stemming* reduces words to approximate root forms by removing suffixes heuristically (e.g., Porter's algorithm [7]).

Definition 2.7 (Lemmatization)**.** *Lemmatization* reduces words to their dictionary form (lemma) using morphological analysis, producing valid words.

Example 2.5 (Stemming vs. Lemmatization)**.**

Word	Porter Stemmer	Lemmatizer
running	run	run
better	better	good
university	univers (wrong)	university
studies	studi	study

Common Pitfall

Do NOT stem for neural models. BERT and sentence-transformers expect natural text. Stemming destroys subword information these models depend on.

2.6.3 Stop Words

Definition 2.8 (Stop Words)**.** *Stop words* are high-frequency words ("the", "a", "is") carrying little discriminative information.

Modern recommendation: keep stop words unless you have a specific reason to remove them. Storage is cheap, stop words matter for phrases ("to be or not to be"), entity names ("The Who"), and neural models.

2.6.4 Preprocessing Depends on Representation

Step	Sparse Retrieval	Dense Retrieval
Tokenization	Custom (vocabulary)	Fixed by model (BPE, WordPiece)
Stemming	Useful	**Not used**
Stop words	Optional	Kept
Truncation	N/A	Critical (context window)

WordPiece Example (BERT):

```
[''un'', ''##touch'', ''##able'']
```

Breaks word into sub-tokens based on statistical frequency.

2.7 Index Compression

Compression is not optional—it makes systems both **faster** and cheaper. CPUs decompress at 2–5 GB/s while SSDs read at ∼0.5 GB/s, so reading compressed data is faster than reading uncompressed data.

2.7.1 Gap Encoding

Definition 2.9 (Gap Encoding)**.** *Gap encoding* (delta encoding) replaces an ordered posting list $[d_1, d_2, \ldots, d_n]$ with gaps $[d_1, d_2 - d_1, d_3 - d_2, \ldots, d_n - d_{n-1}]$. Gaps are typically small, requiring fewer bits [6].

Example 2.6 (Gap Encoding)**. Original:** $[5, 127, 214, 215, 512, 1024, 2048, 10239]$
Gaps: $[5, 122, 87, 1, 297, 512, 1024, 8191]$
Original values need 14 bits each (range up to 10,239). Most gaps are < 256 (8 bits)—much better compression. Decoding is sequential: $d_i = d_{i-1} + \text{gap}_i$.

2.7.2 Variable Byte (VB) Encoding

Definition 2.10 (Variable Byte Encoding)**.** *Variable Byte (VB) encoding* represents each integer using a variable number of bytes: 7 bits per byte for data, 1 bit (MSB) as continuation flag. MSB= 1: last byte; MSB= 0: more bytes follow.

Example 2.7 (VB Encoding Step-by-Step)**. Encode 5:** Binary `101` (3 bits ≤ 7). VB: `1_0000101` (1 byte).
Encode 122: Binary `1111010` (7 bits ≤ 7). VB: `1_1111010` (1 byte).
Encode 824:
1. Binary: `1100111000` (10 bits, needs 2 bytes since $10 > 7$).
2. Split into 7-bit chunks from right: lower = `0111000`, upper = `0000110`.
3. Byte 1 (upper): `0_0000110` (continuation= 0, more follows).
4. Byte 2 (lower): `1_0111000` (continuation= 1, last byte).
5. Result: `00000110 10111000` (2 bytes).

Decode 824:
1. Read bytes until MSB= 1: `00000110`, `10111000`.
2. Strip MSBs: `0000110`, `0111000`.
3. Concatenate: `0000110_0111000` $= 1100111000_2 = 824$.

Compression: Gaps $[5, 122, 87, 1]$ need 4 bytes (VB) vs. 16 bytes (32-bit ints) — **4× compression**.

2.7.3 Compression in Practice

Method	Typical Compression
Variable Byte (VB)	3–4×
PForDelta	6–10×
Dictionary (front coding)	30–50% savings

Production Note

Lucene/Elasticsearch: FOR with SIMD. **Google:** Custom hardware-optimized. **Your projects:** VB encoding is sufficient—simple and effective.

2.8 PyTerrier: Sparse Indexing in Practice

Listing 1: Sparse indexing and BM25 retrieval with PyTerrier [5].

```python
import pyterrier as pt
pt.init()

dataset = pt.get_dataset('irds:vaswani')
indexer = pt.IterDictIndexer("./vaswani_index", overwrite=True)
```

```python
index_ref = indexer.index(dataset.get_corpus_iter())

bm25 = pt.BatchRetrieve(index_ref, wmodel="BM25")
results = bm25.search("aerodynamic flow over wings")
print(results[['docno', 'score', 'rank']].head())

evaluation = pt.Experiment(
    [bm25], dataset.get_topics(), dataset.get_qrels(),
    eval_metrics=["map", "ndcg_cut_10", "P_10"])
print(evaluation)
```

3 Dense Vector Indexing for Neural Retrieval

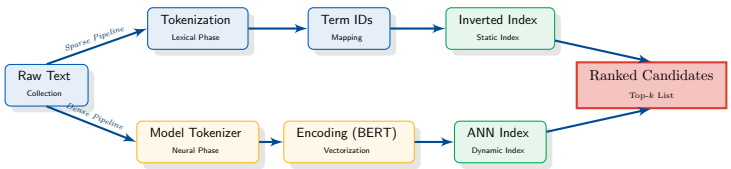
3.1 The Shift to Dense Vectors

Definition 3.1 (Sparse Representation)**.** A *sparse representation* of a document is a vector $\vec{d} \in \mathbb{R}^{|V|}$ over the vocabulary, where most entries are zero. Only terms present in the document have non-zero weights.

Definition 3.2 (Dense Representation (Embedding))**.** A *dense representation* is a vector $\vec{d} \in \mathbb{R}^d$ (typically $d = 768$) produced by a neural encoder (e.g., BERT, sentence-transformers [8]), where **all entries are non-zero** and encode semantic meaning.

	Sparse (BM25)	**Dense (Neural)**
Dimensionality	∼30,000	768
Sparsity	>99.99% zeros	100% non-zero
Matching	Lexical (exact words)	Semantic (meaning)
"car problems" vs. "automobile issues"	No match	Match

Dense vectors cannot use inverted indexes (no discrete terms). We need **Approximate Nearest Neighbor (ANN)** search.

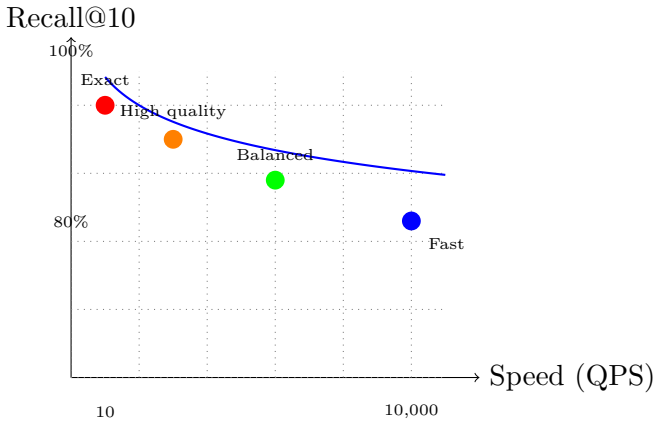


3.2 The ANN Search Problem

Definition 3.3 (Nearest Neighbor Search)**.** Given N vectors $\mathcal{D} = \{\vec{d}_1, \ldots, \vec{d}_N\} \subset \mathbb{R}^d$ and query $\vec{q} \in \mathbb{R}^d$, the k-NN problem finds $\mathcal{S}^* \subset \mathcal{D}$ with $|\mathcal{S}^*| = k$ such that $\forall \vec{d} \in \mathcal{S}^*$, $\forall \vec{d}' \in \mathcal{D} \setminus \mathcal{S}^*$: $\text{sim}(\vec{q}, \vec{d}) \geq \text{sim}(\vec{q}, \vec{d}')$.

Definition 3.4 (Approximate Nearest Neighbor (ANN))**.** ANN search relaxes exactness, returning $\hat{\mathcal{S}}$ such that $|\hat{\mathcal{S}} \cap \mathcal{S}^*|/k \geq 1 - \epsilon$ with high probability, trading recall for speed [1].

Why exact search is infeasible: For $N = 10^9$, $d = 768$: brute-force needs ∼768 billion FLOPs per query (>1s even on GPU).

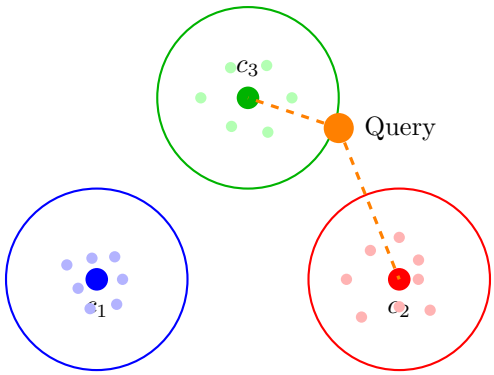


Typical operating points:

- **High quality:** 95–99% recall, 100–500 QPS (production RAG).
- **Balanced:** 90–95% recall, 500–2000 QPS.
- **Fast:** 80–90% recall, 2000–10,000 QPS.

3.3 IVF: Inverted File Index for Vectors

Definition 3.5 (Inverted File Index (IVF))**.** The *IVF* partitions the vector space into k Voronoi cells via k-means, maintaining an inverted list per cluster. At query time, only vectors in the n_{probe} nearest clusters are searched [2].



Search only in 1-2 nearest clusters (not all!)

3.3.1 IVF Construction

Algorithm 1 IVF Construction

Require: Vectors $\mathcal{D}$, number of clusters k

Run k-means on $\mathcal{D} \to$ centers $\{\vec{c}_1, \ldots, \vec{c}_k\}$
for each $\vec{d}_i \in \mathcal{D}$ **do**
  Assign to nearest center: $j^* = \arg\min_j \|\vec{d}_i - \vec{c}_j\|$
end for
Build inverted lists: cluster $j \to$ [vector IDs in cluster j]

3.3.2 IVF Search

Algorithm 2 IVF Search

Require: Query $\vec{q}$, probes n_{probe}, results k

```
Compute distances q⃗ → all centers
Select n_probe nearest centers
candidates ← []
for each selected cluster j do
    Fetch vectors in cluster j; compute exact distances to q⃗
    Append to candidates
end for
return Top-k from candidates by distance
```

Key parameter: n_{probe} (1 = fast/low recall; 100 = slow/high recall).

3.4 Product Quantization (PQ)

Definition 3.6 (Product Quantization)**.** *Product Quantization (PQ)* [2] decomposes a d-dim vector into m subvectors of dimension d/m, independently quantizes each to the nearest centroid from a codebook of K centroids (typically $K = 256$). The vector is stored as m centroid indices (m bytes for $K = 256$).

Example 3.1 (Product Quantization)**.** A 768-dim vector (3072 bytes as float32). With $m = 96$, $K = 256$:

1. **Split** into 96 subvectors of 8 dims each.
2. **Quantize** each subvector to nearest centroid (from 256 centroids per subspace).
3. **Store** 96 centroid IDs $\times$ 1 byte = **96 bytes**.

Compression: $3072/96 = \mathbf{32}\times$.

Distance approximation without decompression:

1. Precompute distance table: query subvector $\to$ all centroids ($96 \times 256 = 24{,}576$ distances, once per query).
2. Per document: $\hat{d}(\vec{q}, \vec{d}) = \sum_{i=1}^{96} \text{distTable}_i[\text{code}_i(\vec{d})]$ — only 96 lookups.

Performance: 32$\times$ memory compression; 10–100$\times$ faster distances; 80–95% recall.

3.4.1 PQ Query Processing in Detail

To understand how PQ answers a query, we must distinguish three phases: *offline training*, *offline encoding*, and *online query processing*. The key innovation is the **Asymmetric Distance Computation (ADC)** trick: the query is *not* quantized—only the database vectors are. This preserves query-side precision while gaining speed on the database side.

Phase 1: Offline Training (Learn Codebooks). Before any documents are indexed, PQ learns m independent codebooks by running k-means on the training data:

1. Collect a representative sample of vectors from the corpus.
2. Split every vector into m subvectors of dimension $d^* = d/m$.
3. For each subspace $i \in \{1, \ldots, m\}$, run k-means on all i-th subvectors to learn K centroids. Denote the codebook for subspace i as $\mathcal{C}_i = \{c_i^{(1)}, c_i^{(2)}, \ldots, c_i^{(K)}\}$, where each $c_i^{(j)} \in \mathbb{R}^{d^*}$.

Result: m codebooks, each containing K centroids of dimension d^*.

Phase 2: Offline Encoding (Compress Database). Each database vector $\vec{d}$ is encoded as a tuple of m centroid indices:

1. Decompose $\vec{d}$ into subvectors: $\vec{d} = [\vec{d}^{(1)}, \vec{d}^{(2)}, \ldots, \vec{d}^{(m)}]$.
2. For each subspace i, find the nearest centroid: $\text{code}_i(\vec{d}) = \arg\min_{j \in \{1,\ldots,K\}} \|\vec{d}^{(i)} - c_i^{(j)}\|^2$.
3. Store the PQ code: $\text{PQ}(\vec{d}) = [\text{code}_1(\vec{d}),\ \text{code}_2(\vec{d}),\ \ldots,\ \text{code}_m(\vec{d})]$.

Each code is an integer in $\{1, \ldots, K\}$, requiring $\lceil \log_2 K \rceil$ bits. For $K = 256$, each code is 1 byte, so the full PQ code is m bytes.

Phase 3: Online Query Processing (ADC Search).

Definition 3.7 (Asymmetric Distance Computation (ADC))**.** In ADC, the squared Euclidean distance between the *exact* query $\vec{q}$ and the *quantized* database vector $\hat{\vec{d}}$ (reconstructed from PQ codes) is approximated as:

$$\|\vec{q} - \hat{\vec{d}}\|^2 = \sum_{i=1}^{m} \|\vec{q}^{(i)} - c_i^{(\text{code}_i(\vec{d}))}\|^2 \tag{2}$$

This is *asymmetric* because $\vec{q}$ is exact while $\vec{d}$ is quantized. The key insight is that each inner term $\|\vec{q}^{(i)} - c_i^{(j)}\|^2$ depends only on the query subvector and the centroid index, so it can be **precomputed into a lookup table**.

The online query processing proceeds in two steps:

Step 1: Build the Distance Lookup Table. For each subspace $i \in \{1, \ldots, m\}$ and each centroid $j \in \{1, \ldots, K\}$, compute:

$$\text{DT}[i][j] = \|\vec{q}^{(i)} - c_i^{(j)}\|^2 \tag{3}$$

This produces an $m \times K$ table. The cost is $O(m \cdot K \cdot d^*)$ floating-point operations, computed **once per query**. For $m = 96$, $K = 256$, $d^* = 8$: this is $96 \times 256 \times 8 = 196{,}608$ FLOPs—negligible.

Step 2: Scan and Accumulate. For each database vector (represented by its PQ code $[\text{code}_1, \ldots, \text{code}_m]$), compute the approximate distance by summing table lookups:

$$\hat{d}(\vec{q}, \vec{d}) = \sum_{i=1}^{m} \text{DT}[i][\text{code}_i(\vec{d})] \tag{4}$$

This requires only m table lookups and $m - 1$ additions per document—no floating-point multiplications at all.

Algorithm 3 PQ Query Processing (Asymmetric Distance Computation)

Require: Query vector $\vec{q} \in \mathbb{R}^d$, PQ codes for N database vectors, codebooks $\{\mathcal{C}_1, \ldots, \mathcal{C}_m\}$, desired results k

Ensure: Top-k nearest neighbors (approximate)

```
// Step 1: Build distance lookup table (once per query)
Decompose q⃗ into subvectors: q⃗^(1), q⃗^(2), ..., q⃗^(m)
for i = 1 to m do
    for j = 1 to K do
        DT[i][j] ← ||q⃗^(i) − c_i^(j)||^2                    ▷ squared L2 in R^{d*}
    end for
end for
// Step 2: Scan all PQ codes, accumulate distances
for each document n = 1 to N do
    dist[n] ← 0
    for i = 1 to m do
        dist[n] ← dist[n] + DT[i][code_i^(n)]               ▷ one table lookup
    end for
end for
// Step 3: Return top-k by smallest distance
return k documents with smallest dist[n]
```

Remark 3.1 (Complexity Analysis)**.**

- **Table construction (Step 1):** $O(m \cdot K \cdot d^*)$. For $m = 96$, $K = 256$, $d^* = 8$: $\sim$197K FLOPs.
- **Distance scan (Step 2):** $O(N \cdot m)$ **integer** lookups + additions. For $N = 1$M, $m = 96$: 96M lookups.
- **Exact brute-force:** $O(N \cdot d)$ **float** multiply-adds. For $N = 1$M, $d = 768$: 768M FLOPs.
- **Speedup:** PQ replaces $d = 768$ float multiply-adds per vector with $m = 96$ integer table lookups—roughly 8× fewer operations, each cheaper. In practice: 10–100× wall-clock speedup.

3.4.2 Worked Example: PQ Query Processing

Example 3.2 (PQ Query Processing, End-to-End)**.** We use a small example for clarity: $d = 8$, $m = 4$ subvectors ($d^* = 2$), $K = 3$ centroids per subspace.

Codebooks (learned offline via k-means).

	$j = 0$	$j = 1$	$j = 2$
$\mathcal{C}_1$	$(1.0,\ 0.0)$	$(0.0,\ 1.0)$	$(-1.0,\ 0.0)$
$\mathcal{C}_2$	$(0.5,\ 0.5)$	$(-0.5,\ 0.5)$	$(0.0,\ -1.0)$
$\mathcal{C}_3$	$(1.0,\ 1.0)$	$(-1.0,\ -1.0)$	$(0.0,\ 0.0)$
$\mathcal{C}_4$	$(0.3,\ 0.7)$	$(-0.3,\ 0.7)$	$(0.8,\ -0.5)$

Database PQ Codes (computed offline).

Doc	code_1	code_2	code_3	code_4
Doc A	0	1	2	0
Doc B	1	0	0	2
Doc C	2	2	1	1

Doc A's code $[0, 1, 2, 0]$ means: subvector 1 was nearest to $\mathcal{C}_1[0] = (1.0, 0.0)$; subvector 2 to $\mathcal{C}_2[1] = (-0.5, 0.5)$; etc. Storage: 4 bytes/doc vs. 32 bytes for the raw 8-dim float32 vector.

Query: $\vec{q} = [0.9,\ 0.1,\ 0.4,\ 0.6,\ -0.8,\ -0.9,\ 0.2,\ 0.8]$. Decompose:

$$\vec{q}^{(1)} = (0.9, 0.1), \quad \vec{q}^{(2)} = (0.4, 0.6), \quad \vec{q}^{(3)} = (-0.8, -0.9), \quad \vec{q}^{(4)} = (0.2, 0.8)$$

Step 1: Build Distance Table. Compute $\text{DT}[i][j] = \|\vec{q}^{(i)} - c_i^{(j)}\|^2$:

Subspace 1 ($\vec{q}^{(1)} = (0.9, 0.1)$):

$$\text{DT}[1][0] = (0.9-1.0)^2 + (0.1-0.0)^2 = 0.01 + 0.01 = \mathbf{0.02}$$
$$\text{DT}[1][1] = (0.9-0.0)^2 + (0.1-1.0)^2 = 0.81 + 0.81 = \mathbf{1.62}$$
$$\text{DT}[1][2] = (0.9+1.0)^2 + (0.1-0.0)^2 = 3.61 + 0.01 = \mathbf{3.62}$$

Subspace 2 ($\vec{q}^{(2)} = (0.4, 0.6)$):

$$\text{DT}[2][0] = (0.4-0.5)^2 + (0.6-0.5)^2 = \mathbf{0.02}$$
$$\text{DT}[2][1] = (0.4+0.5)^2 + (0.6-0.5)^2 = \mathbf{0.82}$$
$$\text{DT}[2][2] = (0.4-0.0)^2 + (0.6+1.0)^2 = \mathbf{2.72}$$

Subspace 3 ($\vec{q}^{(3)} = (-0.8, -0.9)$):

$$\text{DT}[3][0] = (-0.8-1.0)^2 + (-0.9-1.0)^2 = 3.24 + 3.61 = \mathbf{6.85}$$
$$\text{DT}[3][1] = (-0.8+1.0)^2 + (-0.9+1.0)^2 = 0.04 + 0.01 = \mathbf{0.05}$$
$$\text{DT}[3][2] = (-0.8)^2 + (-0.9)^2 = 0.64 + 0.81 = \mathbf{1.45}$$

Subspace 4 ($\vec{q}^{(4)} = (0.2, 0.8)$):

$$\text{DT}[4][0] = (0.2-0.3)^2 + (0.8-0.7)^2 = \mathbf{0.02}$$
$$\text{DT}[4][1] = (0.2+0.3)^2 + (0.8-0.7)^2 = \mathbf{0.26}$$
$$\text{DT}[4][2] = (0.2-0.8)^2 + (0.8+0.5)^2 = 0.36 + 1.69 = \mathbf{2.05}$$

Complete distance table:

	$j = 0$	$j = 1$	$j = 2$
DT[1]	0.02	1.62	3.62
DT[2]	0.02	0.82	2.72
DT[3]	6.85	0.05	1.45
DT[4]	0.02	0.26	2.05

Step 2: Scan PQ Codes (table lookups only). **Doc A** (codes $[0, 1, 2, 0]$):

$$\hat{d}(\vec{q}, A) = \text{DT}[1][\mathbf{0}] + \text{DT}[2][\mathbf{1}] + \text{DT}[3][\mathbf{2}] + \text{DT}[4][\mathbf{0}] = 0.02 + 0.82 + 1.45 + 0.02 = \mathbf{2.31}$$

Doc B (codes $[1, 0, 0, 2]$):

$$\hat{d}(\vec{q}, B) = \text{DT}[1][\mathbf{1}] + \text{DT}[2][\mathbf{0}] + \text{DT}[3][\mathbf{0}] + \text{DT}[4][\mathbf{2}] = 1.62 + 0.02 + 6.85 + 2.05 = \mathbf{10.54}$$

Doc C (codes $[2, 2, 1, 1]$):

$$\hat{d}(\vec{q}, C) = \text{DT}[1][\mathbf{2}] + \text{DT}[2][\mathbf{2}] + \text{DT}[3][\mathbf{1}] + \text{DT}[4][\mathbf{1}] = 3.62 + 2.72 + 0.05 + 0.26 = \mathbf{6.65}$$

Step 3: Rank by distance (ascending). **Doc A** (2.31) < **Doc C** (6.65) < **Doc B** (10.54).

For $k = 1$: return Doc A. Each document required exactly $m = 4$ lookups and 3 additions—*no raw vectors accessed.*

Remark 3.2 (Why This Is Fast)**.** Exact brute-force needs $3 \times 8 = 24$ subtractions, 24 squarings, and 21 additions. PQ needed $3 \times 4 = 12$ lookups and 9 additions. At real scale ($d = 768$, $m = 96$), exact search does 768 FLOPs per document while PQ does 96 lookups—8× fewer, each cheaper. With billions of documents, this is transformative.

Remark 3.3 (Approximation Error)**.** PQ distances are approximate because database vectors are replaced by quantized reconstructions. Error depends on codebook quality. More subvectors (m) and centroids (K) reduce error but increase storage and table size. $m = 96$, $K = 256$ is the standard for 768-dim vectors [2].

3.5 IVF + PQ: The Production Standard

Definition 3.8 (IVF+PQ Index)**.** An *IVF+PQ index* combines IVF (clustering for search-space pruning) with PQ (compression for memory and speed). It is the industry standard for large-scale dense retrieval [3].

Architecture:

1. **Offline:** Cluster vectors (IVF) and compress them (PQ).
2. **Online:** Find n_{probe} nearest clusters, search within using PQ distances, return top-k.

Example 3.3 (FAISS at Scale)**.** 1 billion vectors × 768 dims × 4 bytes = **3 TB** uncompressed. With IVF+PQ (64 bytes/vector): **64 GB** — a **50**× reduction, still 90–95% recall.

3.6 FAISS: Dense Indexing in Practice

Listing 2: Dense indexing with FAISS (IVF+PQ) [3].

```python
import numpy as np, faiss

dimension, n_docs = 768, 100000
doc_emb = np.random.randn(n_docs, dimension).astype('float32')
faiss.normalize_L2(doc_emb)

n_clusters, m_sub, bits = 100, 96, 8
quantizer = faiss.IndexFlatL2(dimension)
index = faiss.IndexIVFPQ(quantizer, dimension, n_clusters, m_sub, bits)
index.train(doc_emb)
index.add(doc_emb)

query = np.random.randn(1, dimension).astype('float32')
faiss.normalize_L2(query)
index.nprobe = 10
distances, indices = index.search(query, k=10)
# Memory: 307 MB uncompressed -> 9.6 MB with IVFPQ (~32x)
```

Table 1: Comparison of sparse and dense indexing.

Aspect	Sparse (Inverted Index)	Dense (IVF+PQ)
Best for	Exact match, rare keywords	Semantic similarity
Recall	Medium (high for entities)	High (semantic)
Precision	Very high	High
Latency	5–20 ms	10–50 ms
Memory	Low	Low–Medium
Scalability	Excellent	Excellent (with IVF)
Explainability	High	Low (black box)

4 Design Decisions: When to Use What

4.1 Decision Matrix

Production Note

Use **sparse** for exact names, model numbers, unique terms. Use **dense** for natural-language questions and synonyms. Use **hybrid** (sparse+dense) for most production RAG systems.

4.2 Case Study: Academic Paper Search (100M+)

Challenge: 100M vectors $\times$ 768 dims $\times$ 4 bytes = 300+ GB; cannot fit raw vectors in RAM.

Solution: Sparse BM25 for titles/authors/abstracts (keyword matching). Dense IVF+PQ: 10k centroids, 32$\times$ PQ compression ($\sim$10 GB, single server). Result: billion-scale search with low latency.

4.3 Common Pitfalls

Common Pitfall

Using only dense for entity-heavy domains. "Python 3.11 release date" may match "snake release from zoo." Use hybrid search.

Common Pitfall

Not tuning ANN parameters. Default n_{probe} gives $\sim$80% recall. Tune on dev set; start at 10 and increase until recall plateaus.

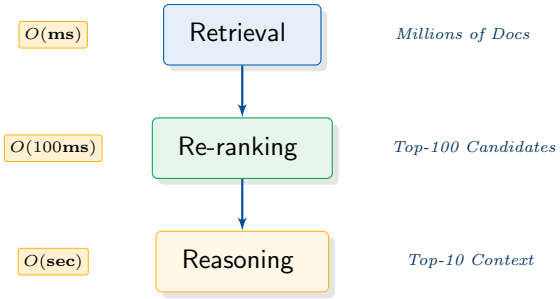
Common Pitfall

Inconsistent preprocessing. Indexing with stemming, querying without $\Rightarrow$ silent failures. Same pipeline for both. For embeddings: no stemming.

Common Pitfall

Forgetting to normalize vectors. Unnormalized embeddings $\Rightarrow$ Euclidean distance $\neq$ cosine similarity. Always: `faiss.normalize_L2(embeddings)`.

4.4 The Full Pipeline



Key Takeaway

Indexing sets the ceiling. Re-ranking and LLM reasoning can only refine what retrieval finds. A missed document is lost forever.

4.5 Production Checklist

Sparse Index:
- ☐ Compression enabled
- ☐ Stop words handled
- ☐ Stemming appropriate
- ☐ Field boosting configured
- ☐ Metadata filterable

Dense Index (IVF+PQ):
- ☐ Vectors normalized
- ☐ n_{probe} tuned
- ☐ Memory budget met
- ☐ PQ training data representative

Monitoring:
- ☐ Track recall@k vs. exact search
- ☐ Monitor p95/p99 latency
- ☐ Alert on index growth
- ☐ Log query/document drift

5 Summary and Next Steps

5.1 What We Covered

1. **Sparse Indexing:** Inverted index structure, text preprocessing, compression (gap + VB encoding), PyTerrier demo.
2. **Dense Indexing:** ANN search, IVF+PQ (clustering + compression), FAISS demo.
3. **Design Decisions:** When to use sparse vs. dense vs. hybrid; scaling; pitfalls.

5.2 Key Takeaways

1. **Inverted indexes are the backbone.** BM25 remains highly competitive for exact/entity queries.
2. **Dense retrieval enables semantic search,** but requires ANN techniques (IVF+PQ) at scale.
3. **Candidate generation is the bottleneck.** Indexing determines what re-rankers/LLMs can see.
4. **Scalability requires compression.** PQ is vital for fitting large indexes in memory.

5.3 Connection to Upcoming Lectures

- **Lecture 3 (Ranking):** BM25 uses inverted indexes; query expansion operates on sparse retrieval.
- **Lecture 4 (Transformer Re-rankers):** Re-rank candidates from today's retrieval stage.
- **Lecture 5 (Dense Retrieval Deep Dive):** Bi-encoders, training; ANN indexing is prerequisite.
- **Lecture 6 (Learning to Rank):** Combine sparse + dense scores; learn fusion weights.

References

[1] Piotr Indyk and Rajeev Motwani. Approximate nearest neighbors: towards removing the curse of dimensionality. In *Proceedings of STOC*, pages 604–613, 1998.

[2] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(1):117–128, 2011.

[3] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.

[4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Proceedings of NeurIPS*, volume 33, pages 9459–9474, 2020.

[5] Craig Macdonald, Nicola Tonellotto, Sean MacAvaney, and Iadh Ounis. Pyterrier: Declarative experimentation in python from bm25 to dense retrieval. In *Proceedings of CIKM*, pages 4526–4533, 2021.

[6] Christopher D Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge university press, 2008.

[7] Martin F Porter. An algorithm for suffix stripping. *Program*, 14(3):130–137, 1980.

[8] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, pages 3982–3992, 2019.

[9] Stephen Robertson, Hugo Zaragoza, et al. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389, 2009.

[10] Gerard Salton, Anita Wong, and Chung-Shu Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620, 1975.